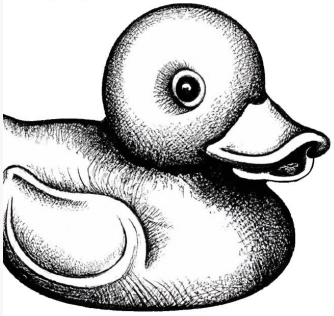
Figuring it out without wasting someone elses time.



Introduction to

Rubber Duck Debugging

O RLY?

Deviant Lycan

Codingame

- Write a bot for a game: "Keep Off The Grass" `https://www.codingame.com/multiplayer/bot-programming/keep-off-the-grass-fall-challenge-2022`.
- For your rating, only your *position in the global ranking* is important:
  - 50%: Reach bronze league.
  - 70%: Reach silver league.
  - 100%: Reach top 30% in silver league.
  - Bonus Point: Reach gold league.
- We will rate your performance after the end of the lecture period, along with the programming tasks.
- You can choose any of the available languages.
- *Important*: let us know your username on Codingame (via Moodle)
- If you like to, you can also add "Friedrich-Schiller-Universität Jena" as your school in your profile (we'll have our own leaderboard).

FRIEDRICH-SCHILLER-
UNIVERSITÄT
JENA

General Training for Programming Competitions & Coding Interviews

November 6, 2024

Arrays and Strings

Markus Fischer, Paul G. Rump, Christoph Staudt

Faculty of Mathematics and Computer Science

Is It Possible to Advance Beyond the Last Index?

Write a program which takes an **array** of n **unsigned integers**, where `A[i]` denotes the **maximum you can advance** from index `i`, and returns whether it is possible to **advance past the last index** starting from the beginning of the array.
(Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)

Draw an **example**:

Is It Possible to Advance Beyond the Last Index?

Write a program which takes an **array** of n **unsigned integers**, where `A[i]` denotes the **maximum you can advance** from index `i`, and returns whether it is possible to **advance past the last index** starting from the beginning of the array.
(Requirements: $\mathcal{O}(1)$ space, $\mathcal{O}(n)$ time)

Draw an **example**: Let `A=[1,2,1]`. We start at the first index with `A[0]=1`, therefore we can proceed one cell. At the second index, we find `A[1]=2`, therefore we can advance two indices – and we arrive behind the last index!

Is It Possible to Advance Beyond the Last Index?

It is natural to try **advancing as far as possible** in each step. But this approach **does not** always work!

Another example: consider the array `B = [2,2,0]`. Here, advancing two steps from the first index leads us to the third element, which is zero – we cannot proceed from here. But going only one step to the second index allows us to then advance another two steps.

Always advancing as far as possible might **skip large entries**.

Check if It Is Possible to Advance Beyond the Last Index in $\mathcal{O}(n)$ Time and With $\mathcal{O}(1)$ Space

can_go_beyond_last_index.cpp

```cpp
bool can_go_beyond_last_index(const vector<size_t> &A) {
  // track the furthest index we know we can advance to
  size_t furthest = 0;
  // stop for-loop, if i is larger than the farthest we can go
  // stop for-loop, if furthest is >= than the array size
  for (size_t i = 0; i <= furthest && furthest < A.size(); ++i) {
    // the furthest we can advance from index i is "i + A[i]"
    // if "i + A[i]" is further than furthest, we update furthest
    furthest = max(furthest, i + A[i]);
  }
  return furthest >= A.size();
}
```

$< 3, 2, 0, 0, 2, 0, 1 >$ × $< 3, 3, 2, 0, 2, 0, 0 >$ × $< 2, 1, 1, 3, 0, 2, 0 >$ ✓

Strings

- Similar to arrays, string problems often have **simple brute-force solutions** that use $\mathcal{O}(n)$ extra space, but subtler in-place solutions that **reduce space complexity** to $\mathcal{O}(1)$.
- Strings are **dynamically allocated**. **Reserving space** in a string reduces the **overhead of reallocation**.

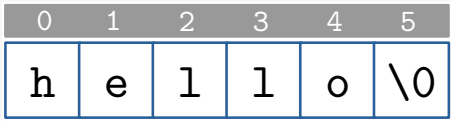
$\rightarrow$ Working in-place can also increase speed!

Strings

- Similar to arrays, string problems often have **simple brute-force solutions** that use $\mathcal{O}(n)$ extra space, but subtler in-place solutions that **reduce space complexity** to $\mathcal{O}(1)$.
- Strings are **dynamically allocated**. **Reserving space** in a string reduces the **overhead of reallocation**.

$\rightarrow$ Working in-place can also increase speed!

- C-style character strings are **terminated** by `'\0'`.
- **C-string** manipulation functions are often **significantly faster** than their C++ version.
- There are **no fast built-in string functions** like replace *string a* with *string b* in C/C++.
- If speed is important, **hardcode** your **regex expressions**.



Convert a String to an Integer

A **string may encode an integer**, for example, “123” encodes 123. Implement a method that takes a **string** representing an integer and returns the corresponding **integer**. Your code should handle negative integers. You cannot use library functions like `std::stoi`. Assume that entered strings can always be converted to a valid integers.

Convert a String to an Integer

A **string may encode an integer**, for example, “123” encodes 123. Implement a method that takes a **string** representing an integer and returns the corresponding **integer**. Your code should handle negative integers. You cannot use library functions like `std::stoi`. Assume that entered strings can always be converted to a valid integers.

string_to_int.cpp

```cpp
// function seems to be significantly faster than std::stoi
inline int string_to_int(const string &s) {
  bool is_negative = s[0] == '-';
  int result = 0;
  // if integer is negative start with index i = 1
  for (size_t i = is_negative; i < s.size(); ++i) {
    result = result * 10 + s[i] - '0';
  }
  return is_negative ? -result : result;
}
```

Convert an Integer to a String

Convert a **signed integer** to a string without using **library functions** like `std::to_string`.

int_to_string.cpp

```cpp
// even this suboptimal function seems to be faster than std::to_string
inline string int_to_string(int num) {
  bool is_negative = num < 0;
  string s;
  do {
    s += char(abs(num % 10)) + '0'; // need abs if num is negative
    num /= 10;
  } while (num);
  // adds the negative sign back
  if (is_negative) {
    s += '-';
  }
  // reverse string
  for (size_t i = 0; i < s.size() / 2; ++i) {
    swap(s[i], s[s.size() - 1 - i]);
  }
  return s;
}
```

Unicode

In **Extended ASCII**, we have **256 characters**. Therefore, a single **one-byte** (8-bit) character is enough to store all of them. If we are going to have **more than 256 characters**, however, we must use **more than one byte** to store their numerical values after 255.

Unicode

In **Extended ASCII**, we have **256 characters**. Therefore, a single **one-byte** (8-bit) character is enough to store all of them. If we are going to have **more than 256 characters**, however, we must use **more than one byte** to store their numerical values after 255.

Different **UTF** (Unicode Transformation Format) **encodings**:

- **UTF-8**: **variable-sized** encoding; uses certain bits in the first byte of the character to denote the number of actual bytes that should be read to retrieve the character fully; UTF-8 is a **superset of ASCII** (128 characters)
- **UTF-16**: uses one or two words (each word has 16 bits within) for storing all characters; hence it is also a **variable-sized encoding**
- **UTF-32**: uses **exactly 4 bytes** for storing the values of all characters (even for ASCII characters); therefore, it is a **fixed-sized encoding**

UTF-8 and UTF-16 are suitable for the applications in which a smaller number of bytes should be used for more frequent characters. UTF-32 consumes more memory compared to others, but it requires less computation power.

C vs. C++ String Processing Speed

count_word.cpp alice_in_wonderland.txt

```cpp
// counts the number of occurrences of a word in the text
size_t count_word_with_cpp(const string &text, const string &word) {
  size_t count_cpp = 0;
  if(word.empty()) return count_cpp;
  // find locates a substring and returns the position to the first occurrence
  size_t pos = text.find(word);
  while (pos != string::npos) { // repeat till the end is reached
    count_cpp++;
    pos = text.find(word, pos + word.size());}
  return count_cpp;
}
```

C vs. C++ String Processing Speed

count_word.cpp alice_in_wonderland.txt

```cpp
// C version is five times faster than the C++ version
size_t count_word_with_c(const string &text, const string &word) {
  size_t count_c = 0;
  if(word.empty()) return count_c;
  // strstr locates a substring and returns a pointer to the first occurrence
  const char *pch = strstr(text.data(), word.data());
  while (pch != nullptr) { // repeat till the end is reached
    count_c++;
    pch = strstr(pch + word.size(), word.data());}
  return count_c;
}
```

Try to Do as Much as Possible In-Place

When you clean up a string, try to replace inconsistent words **in-place** with words that are **not longer**.

When the string size increases, **reallocation** might be necessary $\rightarrow$ **slow**!

my fancy power-shot cam $\rightarrow$ my fancy powershot cam
new power shot camera $\rightarrow$ new powershot camera
2 years old canon-powershot for sale $\rightarrow$ 2 years old powershot for sale

At the end, when you are finished with cleaning and **need** a **well formatted string**, remove **unnecessary spaces**.

Code for In-Place String Replacement

replace_space_out.cpp

```cpp
// function replaces "search_for" in-place with "replace_with" in string "s"
inline void replace_space_out(std::string &s, const std::string &search_for,
                              const std::string &replace_with) {
  // not allowed to replace with a longer string
  if (replace_with.size() > search_for.size()) {
    throw std::runtime_error(
        "size of <replace_with> must be <= size of <search_for>");
  }
  if (search_for.empty()) return;

  // strstr locates a substring and returns a pointer to the first occurrence
  char *pch = const_cast<char *>(strstr(s.data(), search_for.data()));
  while (pch != nullptr) { // repeat till the end is reached
    // copies all characters from replace_with to pch
    strncpy(pch, replace_with.data(), replace_with.size());
    // overwrite invalid characters with spaces
    for (size_t i = 0; i < search_for.size() - replace_with.size(); ++i) {
      pch[i + replace_with.size()] = ' ';
    }
    // search for the next occurrence of string search_for
    pch = const_cast<char *>(strstr(pch + search_for.size(), search_for.data()));
  }
}
```

Review: RegEx

- A regular expression (regex) is a sequence of characters that define a search pattern.
- A useful tutorial: `https://www.geeksforgeeks.org/write-regular-expressions/`
- Regex allows literals (`abc123`), repetitions (`*`, `+`, `{...}`), character classes (`\s`, `\w`, `\d`, ...) and other symbols.

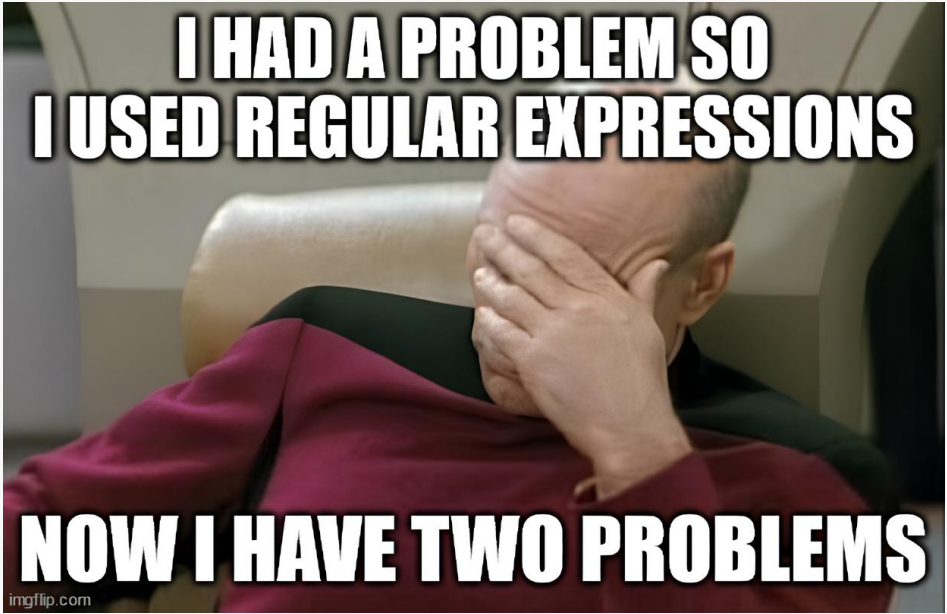
Review: RegEx

- A regular expression (regex) is a sequence of characters that define a search pattern.
- A useful tutorial: `https://www.geeksforgeeks.org/write-regular-expressions/`
- Regex allows literals (`abc123`), repetitions (`*`, `+`, `{...}`), character classes (`\s`, `\w`, `\d`, ...) and other symbols.
- What does the following expression match?
  `^([a-zA-Z0-9_\-\.]+)@([a-zA-Z0-9_\-\.]+)\.([a-zA-Z]2,5)$`

Review: RegEx

- A regular expression (regex) is a sequence of characters that define a search pattern.
- A useful tutorial: `https://www.geeksforgeeks.org/write-regular-expressions/`
- Regex allows literals (`abc123`), repetitions (`*`, `+`, `{...}`), character classes (`\s`, `\w`, `\d`, ...) and other symbols.
- What does the following expression match?
  `^([a-zA-Z0-9_\-\.]+)@([a-zA-Z0-9_\-\.]+)\.([a-zA-Z]2,5)$`

→ It matches e-mail addresses.


I HAD A PROBLEM SO
I USED REGULAR EXPRESSIONS
NOW I HAVE TWO PROBLEMS
imgflip.com

Regex Speed

regex_speed.cpp alice_in_wonderland.txt

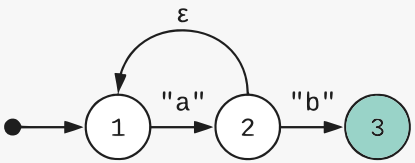
```cpp
void regex_replace(string &text) {
  // if there is a "but" or "and" or "dog" replace it with "cat"
  // creating regex patterns is expensive --> make static const
  static const auto pattern = std::regex{"but|and|dog"};
  static const auto replacement = "cat";
  text = regex_replace(text, pattern, replacement);
}

// semantically the same, but runs 100 times faster
void inplace_replace(string &text) {
  replace_space_out(text, "but", "cat");
  replace_space_out(text, "and", "cat");
  replace_space_out(text, "dog", "cat");
}
```

Regular expressions are a very **powerful but slow** tool for text processing.

Regular Expressions Behind the Scenes

Regular expressions provide a way to **simply** and **tersely describe patterns** in text (e.g. a U.S. postal code such as TX 77845, or an ISO-style date, such as 2009-06-07)

Regular expressions are **compiled** into **state machines** at **run time**.



Here, we have the state machine for the pattern "a+b" (+ means one or more). An epsilon transition (or ε) changes the state without consuming any input characters.

To **increase speed** we can **hardcode simple regex state machines** with basic **while for if else** syntax.

Let's Hardcode "a+b" and Compare the Speed

hardcoded_regex.cpp

```cpp
// hardcoded version of "a+b" regex
// extract all matches into a vector of strings in O(n) time
// runs 20 times faster than the C++ regex version
vector<string> get_all_matches(const string &s) {
  vector<string> result;
  size_t i = 0, j;
  while (i < s.size()) {
    // while character is not an 'a' loop over the string
    while (i < s.size() && s[i] != 'a') { i++; }
    j = i + 1;
    // while character is an 'a' loop over the string
    while (j < s.size() && s[j] == 'a') { j++;}
    if (j < s.size() && s[j] == 'b') { // test if character is a 'b'
      j++;
      // extract match into a new string and push it to result vector
      // we could also work here in-place with the match ...
      result.push_back(s.substr(i, j - i));
    }
    i = j;
  }
  return result;
}
```

Tuning Hardcoded Regex Expressions

Every regex expression offers different possibilities to implement it efficiently. For example, if the regex **contains a repeating, non-changing character sequence**, searching for that character sequence and then checking if the rest of the regex pattern matches may increase speed. (Use the C function `strstr` to search.)

// `\l` is (usually) a lowercase char, `\d` is a decimal digit
'\l\l-file\d{1,4}.txt'

Tuning Hardcoded Regex Expressions

Every regex expression offers different possibilities to implement it efficiently. For example, if the regex **contains a repeating, non-changing character sequence**, searching for that character sequence and then checking if the rest of the regex pattern matches may increase speed. (Use the C function `strstr` to search.)

// `\l` is (usually) a lowercase char, `\d` is a decimal digit
'\l\l-file\d{1,4}.txt' $\rightarrow$ search for -file or maybe .txt first

// {`n`,`m`} means at least n and at most m times, `*` means zero or more
'A{3}B{5,10}C*'

Tuning Hardcoded Regex Expressions

Every regex expression offers different possibilities to implement it efficiently. For example, if the regex **contains a repeating, non-changing character sequence**, searching for that character sequence and then checking if the rest of the regex pattern matches may increase speed. (Use the C function `strstr` to search.)

// `\l` is (usually) a lowercase char, `\d` is a decimal digit
'\l\l-file\d{1,4}.txt' → search for -file or maybe .txt first

// {`n`,`m`} means at least n and at most m times, `*` means zero or more
'A{3}B{5,10}C*' → search for BBBBB first

Try to **exploit the regularities** in an expression when hardcoding. The **structure** of the text is also important.

Fast Conversion to Lower Case

lower_fast.cpp

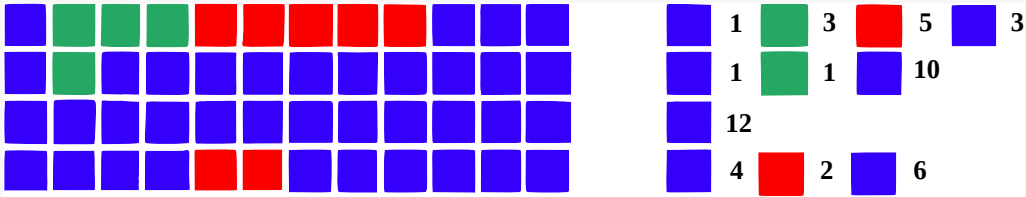
```cpp
// convert char array to lower case
void lower(char *s) {
  // 'a' is greater than 'A' in ASCII
  constexpr char addend = 'a' - 'A';
  // compute strlen
  const size_t n = strlen(s);
  for (size_t i = 0; i < n; i++) {
    // no conditional if
    // compiler is able to use vector instructions
    bool res = 'A' <= s[i] && s[i] <= 'Z';
    s[i] += addend * res; // or use += addend & (0 - res)
  }
}
```

Avoid branches to help the compiler **vectorize** code.

Run-Length Encoding (Popular Interview Question)

Run-length encoding (RLE) compression offers a fast way to do efficient on-the-fly compression and decompression of strings. The idea is simple – **encode successive repeated characters** by the **repetition count and the character**. For example, the RLE of "aaaabcccaa" is "4a1b3c2a". The decoding of "3e4f2e" returns "eeefffffee".

Implement run-length encoding and **decoding** functions. **Assume** the **string to be encoded** consists of **letters of the alphabet**, with **no digits**, and the string **to be decoded** is a **valid encoding**. (Requirements: $\mathcal{O}(n)$ time)



Run-Length Encoding Code

run_length_encoding.cpp

```cpp
string encode(const string &s) {
  string result;
  for (size_t i = 1, count = 1; i <= s.size(); ++i) {
    // found new character so write the count of previous character
    if (i == s.size() || s[i] != s[i - 1]) {
      result += to_string(count) + s[i - 1];
      count = 1;
    } else {  // s[i] == s[i - 1].
      ++count;
    }
  }
  return result;
}
```

Run-Length Decoding Code

run_length_encoding.cpp

```cpp
string decode(const string &s) {
  int count = 0;
  string result;
  for (const char &c : s) {
    if (isdigit(c)) {
      count = count * 10 + c - '0'; // c is a letter of alphabet
    } else {
      result.append(count, c);  // appends count copies of c to result
      count = 0;
    }
  }
  return result;
}
```

Exercises (Upload Solutions to your Repository)

1. Write a function to compute the **minimum number of steps** needed to advance beyond the last index of an array.

   `min_steps_beyond_last_index.cpp`

2. Implement a function that returns the **state** of a **Sudoku board**. 

   `compute_sudoku_state.cpp`

3. Find the **longest river** in a **text** by **changing the line width**. 

   `go_with_the_flow.tar.gz`

4. **Hardcode** two **regex expressions**.

   `hardcode_two_regexes.tar.gz`

Exam Questions

- Outline an **algorithm** to convert a **string to** an **unsigned integer** (no negative numbers).
- Outline an **algorithm** to convert an **unsigned integer** to a **string**.
- Explain the **characteristics** of the character encodings **ASCII**, **Extended ASCII**, **UTF-8**, **UTF-16** and **UTF-32**.
- What **advice** can you give to **modify strings as quickly as possible**?
- **Why** are **hardcoded regex** expressions often **much faster**?
- **Is run-length encoding** a **good compression method** for strings?

4411

Locked. There are disputes about this answer's content being resolved at this time. It is not currently accepting new interactions.

You can't parse [X]HTML with regex. Because HTML can't be parsed by regex. Regex is not a tool that can be used to correctly parse HTML. As I have answered in HTML-and-regex questions here so many times before, the use of regex will not allow you to consume HTML. Regular expressions are a tool that is insufficiently sophisticated to understand the constructs employed by HTML. HTML is not a regular language and hence cannot be parsed by regular expressions. Regex queries are not equipped to break down HTML into its meaningful parts. so many times but it is not getting to me. Even enhanced irregular regular expressions as used by Perl are not up to the task of parsing HTML. You will never make me crack. HTML is a language of sufficient complexity that it cannot be parsed by regular expressions. Even Jon Skeet cannot parse HTML using regular expressions. Every time you attempt to parse HTML with regular expressions, the unholy child weeps the blood of virgins, and Russian hackers pwn your webapp. Parsing HTML with regex summons tainted souls into the realm of the living. HTML and regex go together like love, marriage, and ritual infanticide. The <center> cannot hold it is too late. The force of regex and HTML together in the same conceptual space will destroy your mind like so much watery putty. If you parse HTML with regex you are giving in to Them and their blasphemous ways which doom us all to inhuman toil for the One whose Name cannot be expressed in the Basic Multilingual Plane, he comes. HTML-plus-regexp will liquify the nerves of the sentient whilst you observe, your psyche withering in the onslaught of horror. Regẽx-based HTML parsers are the cancer that is killing StackOverflow *it is too late it is too late we cannot be saved* the transgression of a chi͡ld ensures regex will consume all living tissue (except for HTML which it cannot, as previously prophesied) *dear lord help us how can anyone survive this scourge* using regex to parse HTML has doomed humanity to an eternity of dread torture and security holes *using rege*x as a tool to process HTML establishes a brea*ch between this world* and the dread realm of c͒ͪo͛ͫrrupt entities (like SGML entities, but *more corrupt) a mere glimp*se of the world of reg**ex parsers for HTML will ins**tantly transport a p*rogrammer's consciousness i*nto a wor*l*d of ceaseless screaming, he comes~~, the pestilent sl~~ithy regex-infection wil**l devour your HT**ML parser, application and existence for all time like Visual Basic only worse *he comes he com*es *do not fi*ght h**e com̡es, ̕h**is un̨ho͞ly radiańcé de*stro҉ying all enli̍̈́̂̈́ghtenment, HTML tags* ***lea͠ki̧n͘g fr̶ǫm ̡yo͟ur eye͢s̸ ̛l̕ik͏e liq***uid p*ain, the song of re̸gular expre~~ssion parsing~~ will exti*nguish the voices of mor**tal man from the sp***here I can see it can you see ̲͚̖͔̙î̩́t̲͎̩̱͔́̋̀ it is beautiful t*he f`inal snuf`fing o*f the lie***s of Man ALL IS LOŚ͖̩͇̗̪̏̈́T A***LL IS L**OST th*e pon̷y he come*s he c̶̮omes ~~he comes~~ t*he* **ich**or permeates *al*l MY FAC*E MY FACE ᵒh god n***o NO NOO̼O**O NΘ** stop t*he an*̶͑̾̾̅ͫ͏̙̤g͇̫͛͆̾ͫ̑͆l͖͉̗̩̳̟̍ͫͥͨe̠̅s` ͎a̧͈͖r̽̾̈́͒͑e` **n**ot rè̑ͧ̌aͨl̘̝̙̃ͤ͂̾̆ ZA̡͊͠͝LGΌ ISͮ̂҉̯͈͕̹̘̱ TO͇̹̺ͅƝ̴ȳ̳ TH̘Ë͖́̉ ͠P̯͍̭O̚N̐Y̡ H̸̡̪̯ͨ͊̽̅̾̎Ȩ̬̩̾͛ͪ̈́̀́͘ ̶̧̨̱̹̭̯ͧ̾ͬC̷̙̲̝͖ͭ̏ͥͮ͟Oͮ͏̮̪̝͍M̲̖͊̒ͪͩͬ̚̚͜Ȇ̴̟̟͙̞ͩ͌͝S̨̥̫͎̭ͯ̿̔̀ͅ

Have you tried using an XML parser instead?

> **Moderator's Note**
>
> This post is locked to prevent inappropriate edits to its content. The post looks exactly as it is supposed to look - there are no problems with its content. Please do not flag it for our attention.